

doctask — one page

An agentic system that owns a pile of vendor contracts, their amendments, and the invoices raised against them. Built for the SuperDocs Round 2 engineering task.

What it is. It works out what each document is, extracts the facts that matter, notices where the documents disagree, and composes one grounded deliverable — an **Obligation Register** — in which every claim traces to the exact bytes it came from. It then checks that register against a rule pack written as data, and a human approves or rejects every proposed change, item by item, before anything commits.

Who it is for. The person who has to answer “what are we actually obliged to do for this vendor, and which document says so?” — legal ops, procurement, finance. Today that answer costs an afternoon of reading and is stale the moment a new amendment lands.

RESULTS I CAN MEASURE

CLAIM	HOW IT IS MEASURED
Ten invariants written as failing tests before the code	All 10 green
The suite is real, not mock theater	269 passed, 0 failed, against real PostgreSQL
The tests actually test something	mutation_check.py plants 30 realistic defects; all 30 turn the suite red
It runs with no API key	Whole suite and both demo corpora run on the deterministic fake
An update costs like an update	Fact cache keyed (doc_sha256, extractor_version) ; re-ingest re-extracts and re-embeds nothing
Untouched is provably untouched	Entry-level content hashes and byte-equal rendering, not assertion
It knows what it spent	19 provider calls / 3508 in / 354 out on the 6-document demo, per stage, derived from the journal

The mutation number is the one I would defend hardest. A passing test suite proves nothing about the tests; 30 planted defects all dying proves the suite has teeth.

THE TRADE-OFFS, AND WHY I THINK THEY ARE RIGHT

I did not use LangGraph, which the brief suggests. Its checkpointers resumes graph *state*, but a node interrupted mid-flight is re-executed on resume — and that re-pays for the model call inside it. What this system needs is *work*-level idempotency: the thing that must not repeat is the provider call, not the node. That comes from a durable step journal keyed on deterministic step ids. Once that journal exists, the graph layer adds a dependency without adding a capability. **What it costs me:** no free visualisation, no pre-built node ecosystem, and I own retry semantics myself.

Entity resolution is deterministic string matching, not embeddings. Vendor identity feeds entry ids; entry ids feed content hashes; content hashes are how “nothing else changed” is *proven*. A model call anywhere in that chain would make the proof vary between runs. pgvector is used where the question genuinely is semantic — “does this agreement renew automatically” over long prose.

Anti-hallucination is arithmetic, not a prompt. Every fact carries (doc_id, char_start, char_end, quoted_text) . A verifier re-reads source[char_start:char_end] and compares. A fabricated payment term produces a span that does not hold its quote and dies before the register. Rejected facts are *kept*, marked unsupported — because a system that silently drops what it could not verify looks identical, from outside, to one that never hallucinated.

The audit is staged by cost. Four of the five rules in the demo pack are arithmetic and never reach a model; the audit stage spends exactly one call. The judge is *injected* rather than reached for internally, which is what makes “the free stage is provably free” testable instead of aspirational.

WHERE IT BREAKS — STATED DELIBERATELY

- Injection detection is pattern-based**, not semantic. It lands N2 green and is deliberately deterministic — asking a model “is this an injection?” has a model read attacker-controlled text to decide something, which is the shape of problem being defended against. But a novel phrasing outside the pattern list would go unreported. The structural defense still holds in that case; only the reporting would miss.
- The Anthropic adapter is a stub.** The provider seam is real; no live call has ever gone through this system. A capability may be honestly absent, but it must never be present and broken.
- Vendor extraction is a regex** and will be brittle on messy real documents. I found and fixed one bug in it late — names wrapping across a line break carried the newline into the entry id.
- impact_set is hash comparison, not evidence tracing.** Correct for what the non-modification invariant asserts, but an update still reconciles the whole corpus.
- No filesystem watcher and no React UI.** Incremental ingest is built and tested; the watcher is only the trigger. REST and MCP are complete, so the UI would be a third client of operations that already exist.

WHAT I CUT FROM THE ROUND, AND WHY

Task 2 — the assigned Go CLI — is not built. Two independent reasons. I had no SuperDocs account or API key. And the card’s entire value is release engineering: signed archives, checksums, a Homebrew tap, a Scoop manifest, reproducible builds. A Go binary calling four endpoints with none of that is precisely the hollow stage the brief warns against. Go 1.26 is installed on my machine — this was a time-and-credentials cut, not a capability one, and I would rather say that plainly than ship a shell.

I also spent a meaningful part of the final day recovering from a full disk that stopped PostgreSQL, and from discovering late that the project had never been a git repository of its own. Both are my own operational mistakes, and both cost hours I would otherwise have spent on the Go CLI.

AI DISCLOSURE

Built with heavy use of Claude Code. The architecture, the invariant list, the domain choice and every trade-off argued above are mine; most implementation typing and the first drafts of tests were AI-assisted under review. The mutation harness exists precisely because I did not want to trust either of us about whether the tests were real.